
Security Review Report
NM-0451-1030 - VANA Deposit Contract



NETHERMIND
SECURITY

(September 11, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	Actors	4
4.2	The Seeded Deposit Contract	4
4.3	Deployment and State Seeding	4
4.4	Consensus Layer Deposit Contract Switch	5
4.5	Deposit Log Scanning and Node Bookkeeping	5
5	Risk Rating Methodology	6
6	Issues	7
6.1	[Low] A persisted log cursor already beyond the switch block causes deposits of the new contract to be skipped permanently	7
6.2	[Low] Deposit logs in the switch block are permanently skipped when a historical scan ends at the switch block	8
6.3	[Low] The deposit contract switch validation runs before the <code>-contractdeploymentblock</code> override is applied	9
6.4	[Info] New source files are not registered in the Bazel build	10
6.5	[Info] The <code>ConfigToYaml(...)</code> function drops the deposit contract switch fields	10
6.6	[Info] The deposit count is always read from the current contract, even while the chain is below the switch block	11
6.7	[Best Practices] Failures around the contract switch surface only as a generic sync error	11
6.8	[Best Practices] The retired deposit contract validation accepts the zero address	11
7	Documentation Evaluation	12
8	Test Suite Evaluation	13
8.1	Tests Output	13
8.2	Automated Tools	14
8.2.1	AuditAgent	14
9	About Nethermind	15

1 Executive Summary

This document presents the results of a security review conducted by [Nethermind Security](#) for VANA's **L1 Deposit contract migration**.

This was a diff audit focused on reviewing the new native L1 Deposit contract for VANA's network as well as the changes required to the execution client implementation to accommodate the change of the contract. The goal was to ensure backward compatibility with the old deposit contract as well as future-proofing compatibility with upcoming network upgrades. The new deposit contract will be seeded and it is meant to continue the state of the old contract.

Changes made to the execution client and Prysm introduce the concept of a switch block, which points the nodes to the correct **Deposit** contract that must be queried based on the block number that's currently being looked up.

The audit comprises 4772 lines of the code. **The audit was performed using** (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report 8 points of attention, where three are classified as Low and five are classified as Informational and Best Practices. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the system overview. Section 4 discusses the risk rating methodology. Section 5 details the issues. Section 6 discusses the documentation provided by the client for this audit. Section 7 presents the test suite evaluation and automated tools used. Section 8 concludes the document.

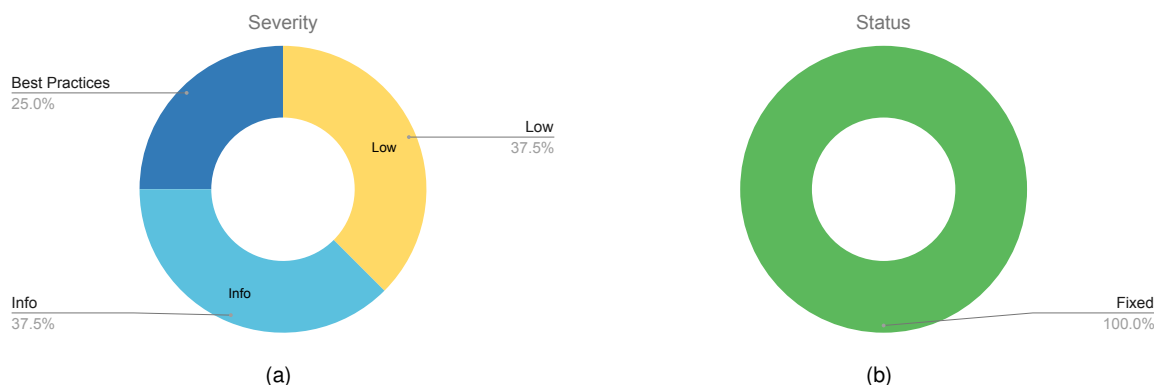


Fig. 1: Distribution of issues: Critical (0), High (0), Medium (0), Low (3), Undetermined (0), Informational (3), Best Practices (2).
Distribution of status: Fixed (8), Acknowledged (0), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	September 02, 2026
Final Report	September 11, 2026
Initial Commit (Deposit Contract)	c24172e
Initial Commit (Prysm Config)	b282da6
Initial Commit (Client Patches)	63a124c
Final Commit (Deposit Contract)	c24172e
Final Commit (Prysm Config)	f8946ff
Final Commit (Client Patches)	63a124c
Documentation Assessment	High
Test Suite Assessment	Medium

2 Audited Files

	File	LoC	Comments	Ratio	Blank	Total
1	/config/params/config.go	635	94	14.8%	92	821
2	/config/params/loader.go	252	11	4.4%	10	273
3	/cmd/flags.go	298	63	21.1%	8	369
4	/cmd/beacon-chain/usage.go	250	4	1.6%	8	262
5	/cmd/beacon-chain/main.go	346	18	5.2%	33	397
6	/cmd/beacon-chain/flags/base.go	312	68	21.8%	9	389
7	/beacon-chain/node/config.go	218	7	3.2%	14	239
8	/beacon-chain/node/node.go	1062	30	2.8%	149	1241
9	/beacon-chain/execution/deposit_contract_switch.go	8	7	87.5%	2	17
10	/beacon-chain/execution/options.go	116	16	13.8%	19	151
11	/beacon-chain/execution/log_processing.go	489	80	16.4%	46	615
12	/beacon-chain/execution/service.go	786	115	14.6%	75	976
13	/script/DeployDepositContractSeeded.s.sol	83	36	43.4%	11	130
14	/contracts/chain/11 Deposit/DepositContractSeeded.sol	192	33	17.2%	29	254
15	/networks/mainnet/config.yml	141	0	0.0%	22	163
	Total	4772	513	10.8%	465	5750

3 Summary of Issues

	Finding	Severity	Update
1	A persisted log cursor already beyond the switch block causes deposits of the new contract to be skipped permanently	Low	Fixed
2	Deposit logs in the switch block are permanently skipped when a historical scan ends at the switch block	Low	Fixed
3	The deposit contract switch validation runs before the <code>--contract-deployment-block override</code> is applied	Low	Fixed
4	New source files are not registered in the Bazel build	Info	Fixed
5	The <code>ConfigToYaml(...)</code> function drops the deposit contract switch fields	Info	Fixed
6	The deposit count is always read from the current contract, even while the chain is below the switch block	Info	Fixed
7	Failures around the contract switch surface only as a generic sync error	Best Practices	Fixed
8	The retired deposit contract validation accepts the zero address	Best Practices	Fixed

4 System Overview

Vana is an EVM-compatible Layer-1 blockchain built around data sovereignty, secured by a Proof-of-Stake consensus layer. The network runs a Geth-based execution client paired with a Prysm-based consensus client, so validators (referred to as *Propagators*) are registered by sending VANA to an L1 deposit contract deployed on the Vana chain itself, which the beacon node treats as its eth1 deposit source. Each accepted deposit is appended as a leaf to an incremental Merkle tree inside the contract, and the beacon node reconstructs that same tree locally from the contract's `DepositEvent` logs in order to vote on `Eth1Data` and activate validators.

The scope of this review covers a migration of that deposit contract. Vana's original deposit contract was retired by upgrading it to a revert-all implementation, which disabled deposits and froze its storage. The migration introduces a new, non-upgradeable replacement contract that reproduces the retired contract's Merkle state, together with the consensus client changes required for a chain to keep a single, continuous deposit history across two different contract addresses.

The following sections describe the components involved and how they interact.

4.1 Actors

- **Validators (Propagators):** The primary participants. A validator generates a BLS key pair, produces the corresponding `DepositData`, and submits a deposit to the active deposit contract. Once the deposit is observed and processed by the consensus layer, the validator is activated and begins proposing and attesting.
- **Deposit Contract Owner:** An `Ownable2Step` owner that controls the deposit contract and can set the minimum accepted deposit amount and whether deposits are restricted to an allowlist. The owner cannot alter the Merkle tree, withdraw funds, or upgrade the contract.
- **Deployer:** The operator who deploys the replacement deposit contract, seeding it from the retired contract's on-chain state.
- **Node Operators:** Operators of the execution and consensus clients. They configure each beacon node with the deposit contract addresses and the block at which the contract switch takes effect, and are responsible for the one-time database bookkeeping step that the migration requires.

4.2 The Seeded Deposit Contract

`DepositContractSeeded` is a port of the canonical Ethereum deposit contract with a single structural change: its constructor seeds **both** the `deposit_count` and the incremental Merkle branch array, rather than starting from an empty tree.

- **State continuity:** For a given count `N`, `get_deposit_root()` folds `branch[h]` for every set bit of `N`, so an identical (`deposit_count`, `branch`) pair yields an identical deposit root. Seeding both values therefore lets a freshly deployed contract reproduce the retired contract's `get_deposit_root()` and `get_deposit_count()` exactly, and subsequent deposits extend the same logical tree. A `get_branch(i)` function is exposed so that a deployment's seeded state can be verified directly.
- **Deposit processing:** `deposit(...)` retains the canonical validation and accounting logic. It checks the pubkey, `withdrawal_credentials`, and signature lengths, requires the value to be a whole number of gwei that fits in a `uint64`, reconstructs the `DepositData` hash tree root and compares it against the user-supplied `deposit_data_root`, emits `DepositEvent`, and folds the new leaf into `branch`.
- **Configurable minimum:** The canonical hardcoded 1 ether minimum amount is replaced by `minDepositAmount`, a parameter that can be set by the **Owner**. As the contract is not upgradeable and the seeded tree is fixed at construction, this and the allowlist state are the only mutable parts of the contract.
- **Restricted mode:** While the `restricted` flag is set, only validator public keys present in the `validators` allowlist may deposit. This mirrors Vana's whitelisting requirement for validators, and the owner may add or remove keys and disable the restriction entirely.
- **Non-upgradeability:** Unlike the previous version, the contract is not upgradeable.

4.3 Deployment and State Seeding

The `DeployDepositContractSeeded` Foundry script performs the migration deployment as a single verified operation.

- **Reading the seed:** At simulation time the script reads the source contract's storage directly, taking slots `0..31` as the `branch` and slot `32` as `deposit_count`. Because the source contract was disabled by upgrading it to a revert-all implementation, its storage is frozen and a read at the latest block is exact. The script requires the source address to hold code and to report a non-zero deposit count.
- **Configuration:** The owner address, minimum deposit, initial `restricted` flag, and an optional list of validator public keys are passed to the constructor.
- **Post-deployment verification:** After broadcasting, the script re-reads the deployed contract and asserts that the deposit count and every branch slot round-tripped correctly. When an expected root is configured, it additionally asserts that `get_deposit_root()` matches the root recorded for the retired contract before it was disabled; otherwise it emits a warning that the root was left unchecked.

4.4 Consensus Layer Deposit Contract Switch

Prysm assumes a single deposit contract for the life of a chain: the address is fixed in the chain configuration, pinned write once in the beacon database, and used as the sole address filter for every deposit log query. The consensus client changes introduce a *switch* logic that describes the contract and the block at which the new contract became authoritative.

- **Configuration:** Two non-spec parameters are added to the beacon chain configuration: `RETIRED_DEPOSIT_CONTRACT_ADDRESS` and `DEPOSIT_CONTRACT_SWITCH_BLOCK`. They are deliberately placed in the chain configuration file rather than the network configuration so that they are identical and are not served over the `/eth/v1/config/spec` endpoint. The `-retired-deposit-contract` and `-deposit-contract-switch-block` flags override them for testing. Leaving both unset disables the switch and preserves the client's prior behaviour.
- **Validation:** `validateDepositContractSwitch(...)` runs at startup and rejects an incoherent switch: the retired address and the switch block must be supplied together, the address must be a valid hex address distinct from the current deposit contract, and the switch block must lie above the configured contract deployment block, since otherwise the retired contract's deposits would never be scanned.
- **Authoritative contract resolution:** The execution service receives both addresses and the switch block through the `WithDepositContractSwitch(...)` option. `depositContractForBlock(...)` then resolves, for any eth1 block, which contract is authoritative for that block, and additionally reports the first block beyond that contract's range. Exactly one contract is authoritative per block: below the switch block only the retired contract may emit deposits, and at or above it only the new one.

4.5 Deposit Log Scanning and Node Bookkeeping

The beacon node's execution service builds its local deposit trie by scanning eth1 blocks for `DepositEvent` logs, either block-by-block while following the chain head or in batches during historical catch-up. Both paths are made switch-aware.

- **Range clamping:** In `processBlockInBatch(...)`, a batch that would span the switch is clamped to end at the last block below it, so a range is never queried against the wrong contract. Because callers resume from the block the batch returns, a clamped batch that starts at that same boundary block advances past the switch once that block has been scanned, ensuring the scan always makes progress.
- **Per-log enforcement:** `ProcessLog(...)` re-checks that a deposit log's emitting address matches the contract authoritative for its block, and fails rather than skipping if it does not. Deposits are folded into the trie in Merkle index order, so an unexpected log means the index sequence can no longer be trusted; failing keeps the retired contract that resumes emitting events from corrupting the tree.
- **Trie construction and persistence:** `ProcessDepositLog(...)` unpacks the log, requires the Merkle index to follow the previously seen one, inserts the deposit hash into the deposit trie, and records the deposit in the deposit cache. Before genesis it also feeds deposits into the pre-genesis state to determine chain start; afterwards, it queues them as pending deposits. The EIP4881 deposit tree is finalized as it advances, and the accumulated execution chain data, including the deposit snapshot, is periodically persisted to the beacon database.
- **Database bookkeeping:** The beacon database pins the deposit contract address on first start and doesn't run against a different one, guarding against accidentally running on the wrong network. Because a legitimate migration changes that address, the new `-clear-deposit-contract` flag removes only the stored address, so that it is recorded again from the current configuration on the same start, leaving all other data intact. This preserves the write-once semantics of the guard while avoiding a full database wipe.
- **Network parameters:** The Vana mainnet chain configuration defines the network's deviations from Ethereum mainnet, including a 6 second slot with 8 slots per epoch, an eth1 follow distance of 32 blocks, deposit chain and network IDs of 1480, a minimum deposit of 1,000 VANA and an activation and maximum effective balance of 35,000 VANA, Vana specific fork versions, and far-future epochs for the forks the network does not activate.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Low] A persisted log cursor already beyond the switch block causes deposits of the new contract to be skipped permanently

File(s): [beacon-chain/execution/log_processing.go](#), [beacon-chain/node/node.go](#)

Description: The execution service rebuilds the deposit Merkle tree by scanning `DepositEvent` logs and persists the last scanned execution block in `LastRequestedBlock`. On every startup, and whenever the node falls behind, the `processPastLogs(...)` function resumes scanning from that persisted position and only moves forward. The deposit contract switch selects the retired contract for blocks below `DEPOSIT_CONTRACT_SWITCH_BLOCK` and the current contract at or above it, based solely on the block being scanned.

```

1 func (s *Service) processPastLogs(ctx context.Context) error {
2     // @audit-issue The scan resumes from the persisted cursor regardless of the switch.
3     currentBlockNum := s.latestEth1Data.LastRequestedBlock
4     deploymentBlock := params.BeaconNetworkConfig().ContractDeploymentBlock
5     // ...
6     currentBlockNum = max(currentBlockNum, deploymentBlock)
7     // ...
8 }

```

A node that first synchronised (fresh database or restored snapshot) with a binary or configuration that did not know about the switch persists a cursor that may already be beyond the switch block, because it kept scanning only the retired contract. When such a node is upgraded and reconfigured, `processPastLogs(...)` resumes at that cursor and queries the current contract from there on. Any `DepositEvent` emitted by the current contract between the switch block and the persisted cursor is never requested. The `-clear-deposit-contract` flag introduced as the migration mechanism does not help, because the `clearDepositContractAddress(...)` function removes only the stored contract address and leaves `LastRequestedBlock`, the deposit containers and the tree snapshot untouched.

```

1 func (b *BeaconNode) clearDepositContractAddress() error {
2     // ...
3     // @audit Only the address key is deleted; the scan cursor and deposit state remain.
4     if err := b.db.ClearDepositContractAddress(b.ctx); err != nil {
5         return errors.Wrap(err, "could not clear deposit contract address")
6     }
7     // ...
8 }

```

Once indices are missing, the next deposit of the current contract fails the sequential index check in `ProcessDepositLog(...)` (received incorrect merkle index), the cursor is rolled back and the same range is retried indefinitely. The affected node stops following deposits, votes stale `Eth1Data`, and cannot build blocks that must include deposits once the chain state adopts them. On the Vana mainnet the cursor is persisted only at a node's first start and every 1000 deposits, so the exposed population is limited to nodes whose first start happened after the switch with the old setup, and to snapshots created from such nodes.

Recommendation(s): Consider treating the first activation of a switch as an explicit, persisted migration step: when the stored cursor is at or beyond `DEPOSIT_CONTRACT_SWITCH_BLOCK` and the switch has not been applied before, rewind the cursor to the switch block (or verify that the local tree already contains the current contract's deposits) before resuming. Consider also documenting that snapshots must be regenerated after the switch, and adding a test that starts from a persisted cursor beyond the switch with deposits inside the skipped range.

Status: Fixed

Update from the client: Fixed in the commit [6df9d56](#) and [08bf8d0](#)

Update from the Nethermind Security Team: The fix is incomplete: the applied-switch marker is persisted immediately, while the rewind cursor exists only in memory. The cursor is written to disk only by `savePowchainData(...)`, which runs at every 1000th deposit index, at chain start, and in a first-start branch gated on `chainStartData.GenesisBlock == 0` that cannot fire again on Vana (its genesis eth1 block is 47). For exactly the population this fix targets, a node whose persisted cursor is above the switch, the following sequence re-opens the hole: the first patched start rewinds in memory, durably records the switch as applied, and rescans the missed deposits into memory only; on any later restart before one of the rare persistence points fires, the stale cursor is restored from disk while the migration is skipped because the marker is already present. The deposits between the switch block and the stale cursor are then missed permanently, since the one-shot protection has consumed itself. We recommend persisting the execution chain data with the rewind cursor in the same step that records the applied switch, or recording the marker only once the cursor has durably passed the switch block again. Additionally, we recommend that `applyDepositContractSwitchMigration(...)` refuse to start when a recorded switch differs from the configured one (applied != switchBlock) instead of migrating again. A database that already scanned under one switch block cannot in general be made consistent with a different one by rewinding: blocks between the two values keep their old contract attribution, and current contract deposits between a raised switch and its predecessor pass the deposit-height refusal unnoticed. Since a legitimate second contract switch is not representable in the current configuration (a single retired address and switch block), any mismatch indicates a misconfiguration or a deliberate correction, and both warrant an explicit stop with instructions rather than a silent re-migration. Such a check would also need an escape hatch, for example extending `-clear-deposit-contract` to clear the recorded switch as well.

Update from the client: Fixed in the commit [f8946ff](#)

6.2 [Low] Deposit logs in the switch block are permanently skipped when a historical scan ends at the switch block

File(s): beacon-chain/execution/log_processing.go

Description: The beacon node builds its deposit trie by scanning the execution chain for DepositEvent logs emitted by the deposit contract. Because the chain replaced its deposit contract at a configured switch block, exactly one contract is authoritative for any given block: the depositContractForBlock(...) function from the Service contract returns the retired contract for blocks below depositContractSwitchBlock and the current contract at or above it, together with the first block beyond that contract's range (the boundary).

Catching up on history is done by the processPastLogs(...) function, which walks forward in batches by calling the processBlockInBatch(...) function. Each batch reports back the block it got to, and that number is used as a bookmark meaning "everything up to here has been scanned". When the walk is over, the bookmark is saved as LastRequestedBlock:

```

1  for currentBlockNum < latestFollowHeight {
2      currentBlockNum, batchSize, err = s.processBlockInBatch(ctx, currentBlockNum,
3          latestFollowHeight, batchSize, additiveFactor, logCount, headersMap)
4      if err != nil {
5          return err
6      }
7  }
8
9  s.latestEth1DataLock.Lock()
10 // @audit-issue The bookmark the loop ends on is saved as the resume point, and no code
11 // path ever goes back below it.
12 s.latestEth1Data.LastRequestedBlock = currentBlockNum
13 s.latestEth1DataLock.Unlock()

```

Normally the bookmark is honest, because a batch reports end, the last block it actually asked the execution client about. The code path added for the contract switch is the exception, because two rules meet there:

- a batch must never cross the switch block, since the contract to query differs on either side of it, so end is trimmed down to switchBlock - 1;
- a batch must always move forward, otherwise the loop would repeat the same batch forever;

When a batch happens to start exactly at switchBlock - 1, the two rules collide: trimming leaves end equal to start, so nothing moved. The code resolves this by reporting switchBlock instead — a block it never asked about:

```

1  addr, boundary := s.depositContractForBlock(start)
2  clampedToBoundary := boundary != 0 && end >= boundary
3  if clampedToBoundary {
4      // @audit The batch is trimmed to the last block below the switch.
5      end = boundary - 1
6  }
7  // ...
8  currentBlockNum = end
9  // @audit-issue The bookmark is moved to the switch block, which this batch never
10 // queried against any contract.
11 if clampedToBoundary && currentBlockNum <= start {
12     currentBlockNum = boundary
13 }

```

Most of the time that inaccurate bookmark is harmless. The walk continues, the next batch starts at switchBlock, and the block does get scanned after all. It turns into a real hole only when the walk stops right at that point, which is the case when the follow height is exactly switchBlock: every batch is then trimmed to switchBlock - 1, so this forward jump is the only thing that can end the loop. It also does not depend on an unlucky batch size. With the follow height sitting on the switch, the trimming walks the position to switchBlock - 1, and the batch after that necessarily starts there and takes the jump.

The loop then ends, switchBlock is saved as if it had been scanned, and normal block-by-block processing picks up at the block after the bookmark. Nothing ever looks at the switch block:

```

1  // @audit-issue Normal processing resumes one block above the saved bookmark, so the
2  // switch block is never scanned.
3  for i := s.latestEth1Data.LastRequestedBlock + 1; i <= requestedBlock; i++ {
4      // ...
5      err = s.ProcessETH1Block(ctx, new(big.Int).SetUint64(i))
6      // ...
7  }

```

This situation is not hypothetical, because the follow height is not a fixed number. The followedBlockHeight(...) function derives it from block timestamps, so it climbs one block at a time and therefore passes through switchBlock, sitting there for about one block's worth of time, roughly one follow distance after the chain produced the switch block. Anything that starts a historical walk during that short window, from a position below the switch, hits the bug. Two things start such a walk: node startup, and the catch-up path taken by the

requestBatchedHeadersAndLogs(...) function once the node has fallen more than maxTolerableDifference blocks behind. A node that restarts or catches up while the switch block is that fresh, for example during a restart or upgrade around the switch lands in this window.

If the switch block happens to contain a DepositEvent, that deposit is dropped without a trace: no error is returned and no warning is logged. The hole can still close by itself, but only while the bookmark still reads exactly switchBlock, because a restart or a catch-up at that point rescans starting from the bookmark, switch block included. Once normal processing has moved on to switchBlock + 1, the bookmark is past the hole and it can never be filled.

Deposits carry consecutive Merkle indices, and the node checks that each one follows the previous. The next deposit therefore arrives with an index one too high and is rejected by the ProcessDepositLog(...) function with received incorrect merkle index if one deposit is missed. That rejection also stops the bookmark from advancing, so the node keeps retrying the same block: deposits stop being ingested for good, the execution service stays in a permanent error state, and the only way out is to resync the deposit history.

The existing switch tests do not cover this, as they all use follow heights well above the switch block (200, 300 and 500 against a switch block of 100), so the walk never has to end on the forward jump.

Recommendation(s): In processPastLogs, after the loop and before storing LastRequestedBlock, adjust the resume point back when it equals the switch block:

```
1 if b := s.cfg.depositContractSwitchBlock; b != 0 && currentBlockNum == b {
2     currentBlockNum = b - 1
3 }
```

At loop exit the value can equal switchBlock via the boundary jump, so the adjustment persists a block that was scanned, and the per-block path then queries the switch block with the new contract's address.

Consider also extending the test suite with a case where the follow height equals the switch block, asserting that every block at or below the saved resume point was queried, so that this exit path is covered.

Status: Fixed

Update from the client: Good point! It has been fixed in the commit [275b9ae](#). The idea of the fix is that we keep the query range (start, end) and determine the corresponding deposit contract address in the range if the switch block is in the range.

6.3 [Low] The deposit contract switch validation runs before the -contractdeployment-block override is applied

File(s): [beacon-chain/node/node.go](#), [beacon-chain/node/config.go](#)

Description: During startup the beacon node applies its CLI overrides in sequence. The configureEth1Config(...) function applies the deposit contract flags and ends by calling validateDepositContractSwitch(...), which rejects a switch block that is not above the contract deployment block, because the historical scan starts at max(cursor, ContractDeploymentBlock) and a switch block at or below it would mean the retired contract is never scanned.

```
1 if err := configureEth1Config(cliCtx); err != nil {
2     return errors.Wrap(err, "could not configure eth1 config")
3 }
4
5 // @audit-issue The deployment block flag is applied only after the validation above ran.
6 configureNetwork(cliCtx)
```

The -contract-deployment-block flag is applied later, inside configureNetwork(...), so the validation compares DEPOSIT_CONTRACT_SWITCH_BLOCK against the pre-override deployment block.

```
1 if deployment := params.BeaconNetworkConfig().ContractDeploymentBlock; switchBlock <= deployment {
2     // @audit `deployment` is the value before the CLI override is applied.
3     return fmt.Errorf(
4         "deposit contract switch block %d must be above the contract deployment block %d, ...",
5         switchBlock, deployment)
6 }
```

As a result a valid configuration can be rejected when the default deployment block is higher than the effective one, and an invalid configuration is accepted when the flag raises the deployment block to or above the switch block; in the latter case the node silently starts scanning at the raised deployment block and never reads the retired contract's history. The unit test TestValidateDepositContractSwitch sets the network config directly, so it does not exercise the real startup order. With Vana's current invocation (-chain-config-file and -contract-deployment-block=0) both values are 0 and the issue has no effect, but the check does not protect against the configuration it was written for.

Recommendation(s): Consider applying configureNetwork(...) before the switch validation, or moving the validation to a final step that runs after every chain and network override has been applied, and adding a startup-level test that supplies all three flags together.

Status: Fixed

Update from the client: Fixed in the commit [0296ce9](#)

6.4 [Info] New source files are not registered in the Bazel build

File(s): [beacon-chain/execution/BUILD.bazel](#)

Description: Prysm builds its release binaries and container images with Bazel, and every Go package lists its source files explicitly in a BUILD.bazel file. The change adds `deposit_contract_switch.go` and `deposit_contract_switch_test.go` to the `beacon-chain/execution` package but does not add them to the `srcs` lists of the `go_library` and `go_test` targets. The `go build` and `go test` commands succeed because they discover files automatically, but the Bazel build of the execution package fails, since `log_processing.go` references the `depositContractForBlock(...)` method that Bazel never compiles.

```
1 bazel build //beacon-chain/execution:go_default_library
2 beacon-chain/execution/log_processing.go:63:15: s.depositContractForBlock undefined
3 beacon-chain/execution/log_processing.go:104:23: s.depositContractForBlock undefined
4 beacon-chain/execution/log_processing.go:403:22: s.depositContractForBlock undefined
```

The dependent `//beacon-chain/node` targets fail as well, and the new switch tests are not part of any Bazel test target. The repository's GitHub workflows run `go test`, so this is not caught by CI.

Recommendation(s): Consider regenerating the Bazel metadata (for example with Gazelle) so that both files appear in their respective `srcs` lists, and consider running the affected Bazel targets in CI so that a passing `go test` cannot mask missing build metadata.

Status: Fixed

Update from the client: Fixed in [f7356e8](#)

6.5 [Info] The `ConfigToYaml(...)` function drops the deposit contract switch fields

File(s): [config/params/loader.go](#)

Description: The change adds `RETIRED_DEPOSIT_CONTRACT_ADDRESS` and `DEPOSIT_CONTRACT_SWITCH_BLOCK` to `BeaconChainConfig` and teaches `UnmarshalConfig(...)` to parse them. The `ConfigToYaml(...)` function, which serialises an active configuration back to YAML and is used by the end-to-end test components to write the config file handed to test nodes, was not updated and still emits only `DEPOSIT_CONTRACT_ADDRESS`.

```
1 lines := []string{
2     // ...
3     // @audit-issue The two switch keys are not emitted.
4     fmt.Sprintf("DEPOSIT_CONTRACT_ADDRESS: %s", cfg.DepositContractAddress),
5     // ...
6 }
```

Any configuration that passes through this function silently loses the switch, so an end-to-end test that configured a switch would exercise the no-switch path without failing. The config comparison helpers used in the params tests also do not compare the two new fields, so the omission is not detected.

Recommendation(s): Consider emitting both fields in `ConfigToYaml(...)`, comparing them in the config equality helpers, and adding a round-trip test starting from non-default switch values.

Status: Fixed

Update from the client: Fixed in the commit [a96f190](#)

6.6 [Info] The deposit count is always read from the current contract, even while the chain is below the switch block

File(s): [beacon-chain/execution/log_processing.go](#), [beacon-chain/execution/rpc_connection.go](#)

Description: The deposit contract switch makes log queries contract-aware: blocks below DEPOSIT_CONTRACT_SWITCH_BLOCK are queried against the retired contract and the rest against the current one. The historical scan in `processPastLogs(...)` additionally reads the total deposit count with an `eth_call` to size its batches. That call is not switch-aware: `depositContractCaller` is bound once to the current contract address in `setupExecutionClientConnections(...)`, and `processPastLogs(...)` calls it unconditionally at every startup and whenever the node falls more than 50 blocks behind.

```

1 // @audit-issue Called at the latest block against the current contract, regardless of the switch.
2 rawLogCount, err := s.depositContractCaller.GetDepositCount(&bind.CallOpts{})
3 if err != nil {
4     return err
5 }

```

If the switch is configured before the current contract is deployed, or if a node points at a wrong address, the call returns no data, `processPastLogs(...)` fails, and `initPOWService(...)` retries indefinitely with the generic message "Unable to process past deposit contract logs, perhaps your execution client is not fully synced". The execution service never finishes initialising, so the node produces no `Eth1Data` votes and cannot include deposits. This is an operational ordering constraint (deploy the contract first, then roll out the configuration) that the code does not express.

Recommendation(s): Consider making the count lookup consistent with the switch, for example by reading it from the contract that is authoritative for the scanned range, or by tolerating a missing current contract while the followed block height is below the switch block, and consider emitting a specific error message when the call fails so operators can distinguish this case from an unsynced execution client.

Status: Fixed

Update from the client: The deposit count should always be read from the current deposit contract. A more specific error message was added in the commit [ee47e3c](#)

6.7 [Best Practices] Failures around the contract switch surface only as a generic sync error

File(s): [beacon-chain/execution/service.go](#), [beacon-chain/execution/log_processing.go](#)

Description: Every failure the switch can produce (an index gap after the boundary, a deposit log from the wrong contract, a missing current contract when the deposit count is read) is returned from `processPastLogs(...)` and handled by the retry loop in `initPOWService(...)`, which logs the rate-limited message "Unable to process past deposit contract logs, perhaps your execution client is not fully synced" and retries forever. There is no log line when the boundary is crossed, no metric for the address guard in `ProcessLog(...)`, and no hint that the switch configuration is involved. Operators debugging a node that stopped following deposits will look at the execution client first.

Recommendation(s): Consider logging at INFO when the scan crosses DEPOSIT_CONTRACT_SWITCH_BLOCK (with the contract addresses and the current index), adding a metric or dedicated error for the address guard, and mentioning the switch configuration in errors raised within one batch of the boundary.

Status: Fixed

Update from the client: Addressed in the commit [767c8a5](#)

6.8 [Best Practices] The retired deposit contract validation accepts the zero address

File(s): [beacon-chain/node/config.go](#)

Description: The `validateDepositContractSwitch(...)` function rejects a malformed retired address and one equal to the current contract, but accepts `0x0000000000000000000000000000000000000000000000000000000000000000`, which `common.IsHexAddress(...)` considers valid. An `eth_getLogs` query against an address without code returns an empty result rather than an error, so a placeholder or mistyped retired address yields an empty history below the switch block without any diagnostic.

Recommendation(s): Consider rejecting the zero address explicitly, in line with the checks already performed on the other fields.

Status: Fixed

Update from the client: Fixed in [b2a6bc4](#)

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Vana's documentation

Vana's team explained to us the challenge of the current engagement during the kickoff call and has addressed all questions and concerns raised by the Nethermind Security team during regular sync calls, providing valuable insights and a comprehensive understanding of the project's technical aspects.

Vana also provided a detailed **migration strategy** document, which outlined each step of the migration process as well as a test suite for the changes being made.

8 Test Suite Evaluation

Remarks about the failing tests

Two tests may fail when running the full suites of the affected packages. Both failures are timing-related, exist in the unmodified upstream code (they fail identically on the base commit 51b5a75eb, without the audited changes), and are unrelated to the deposit contract switch.

- TestProcessLogs_DepositRequestsStarted (beacon-chain/execution): the test setup submits transactions to a simulated eth1 backend, which can reject one with *replacement transaction underpriced* when the package runs under parallel load. The test passes consistently when run in isolation.
- TestState_CanSaveRetrieveValidatorEntriesFromCache (beacon-chain/db/kv): the test reads from a ristretto cache immediately after writing to it, but ristretto's Set is asynchronous, so the read can race the write. Whether it passes depends on scheduling; inserting a Wait() on the cache makes it pass deterministically.

The stack trace printed near the first failure originates from TestHandlePanic_OK, which panics intentionally to exercise the panic recovery handler; that test passes, and the trace is expected output rather than an error.

The tests added by the audited pull request pass deterministically.

8.1 Tests Output

```
> go test ./beacon-chain/execution/... ./beacon-chain/node/... ./beacon-chain/db/kv/... ./cmd/... ./config/params/... -tags develop
↳ -count=1

--- FAIL: TestProcessLogs_DepositRequestsStarted (0.11s)
    assertions.go:158: log_processing_test.go:538 Could not deposit to deposit contract: replacement transaction underpriced
goroutine 78320 [running]:
runtime/debug.Stack()
    /home/ahmed/.asdf/installs/golang/1.26.6/go/src/runtime/debug/stack.go:26 +0x64
runtime/debug.PrintStack()
    /home/ahmed/.asdf/installs/golang/1.26.6/go/src/runtime/debug/stack.go:18 +0x1c
github.com/OffchainLabs/prysm/v7/beacon-chain/execution.safelyHandlePanic()
    /home/ahmed/Nethermind/NM-0451-1030-VANA-DEPOSIT-CONTRACT/NM-0451-1030-Vana-deposit/beacon-chain/execution/service.go:490 +0x104
panic({0x1a2e4e0?, 0x3392260?})
    /home/ahmed/.asdf/installs/golang/1.26.6/go/src/runtime/panic.go:860 +0x12c
github.com/OffchainLabs/prysm/v7/beacon-chain/execution.(*Service).processBlockHeader(0x709755e1db00, 0x0)
    /home/ahmed/Nethermind/NM-0451-1030-VANA-DEPOSIT-CONTRACT/NM-0451-1030-Vana-deposit/beacon-chain/execution/service.go:433 +0x44
github.com/OffchainLabs/prysm/v7/beacon-chain/execution.TestHandlePanic_OK(0x709755d5b448)
    /home/ahmed/Nethermind/NM-0451-1030-VANA-DEPOSIT-CONTRACT/NM-0451-1030-Vana-deposit/beacon-chain/execution/service_test.go:306
↳ +0x208
testing.tRunner(0x709755d5b448, 0x1fe9f38)
    /home/ahmed/.asdf/installs/golang/1.26.6/go/src/testing/testing.go:2036 +0xc4
created by testing.(*T).Run in goroutine 1
    /home/ahmed/.asdf/installs/golang/1.26.6/go/src/testing/testing.go:2101 +0x3a8
FAIL
FAIL    github.com/OffchainLabs/prysm/v7/beacon-chain/execution 27.734s
?      github.com/OffchainLabs/prysm/v7/beacon-chain/execution/testing [no test files]
ok     github.com/OffchainLabs/prysm/v7/beacon-chain/execution/types 0.013s
ok     github.com/OffchainLabs/prysm/v7/beacon-chain/node 10.349s
ok     github.com/OffchainLabs/prysm/v7/beacon-chain/node/registration 0.023s
Migrating state validators to new schema. 100% [=====] [0s:0s]
Migrating state validators to new schema. 100% [=====] [0s:0s]
Migrating state validators to new schema. 100% [=====] [0s:0s]
Migrating state validators to new schema. 100% [=====] [0s:0s]
Migrating state validators to new schema. 100% [=====] [0s:0s]
Migrating state validators to new schema. 100% [=====] [0s:0s]
--- FAIL: TestState_CanSaveRetrieveValidatorEntriesFromCache (0.02s)
    assertions.go:40: state_test.go:330 Values are not equal, want: true (bool), got: false (bool)
    assertions.go:188: state_test.go:331 Unexpected nil value
FAIL
FAIL    github.com/OffchainLabs/prysm/v7/beacon-chain/db/kv 55.982s
ok     github.com/OffchainLabs/prysm/v7/cmd 0.047s
ok     github.com/OffchainLabs/prysm/v7/cmd/beacon-chain 0.216s
?      github.com/OffchainLabs/prysm/v7/cmd/beacon-chain/blockchain [no test files]
?      github.com/OffchainLabs/prysm/v7/cmd/beacon-chain/das [no test files]
?      github.com/OffchainLabs/prysm/v7/cmd/beacon-chain/das/flags [no test files]
?      github.com/OffchainLabs/prysm/v7/cmd/beacon-chain/db [no test files]
ok     github.com/OffchainLabs/prysm/v7/cmd/beacon-chain/execution 0.049s
ok     github.com/OffchainLabs/prysm/v7/cmd/beacon-chain/flags 0.024s
?      github.com/OffchainLabs/prysm/v7/cmd/beacon-chain/genesis [no test files]
ok     github.com/OffchainLabs/prysm/v7/cmd/beacon-chain/jwt 0.048s
ok     github.com/OffchainLabs/prysm/v7/cmd/beacon-chain/storage 0.080s
?      github.com/OffchainLabs/prysm/v7/cmd/beacon-chain/sync/backfill [no test files]
?      github.com/OffchainLabs/prysm/v7/cmd/beacon-chain/sync/backfill/flags [no test files]
?      github.com/OffchainLabs/prysm/v7/cmd/beacon-chain/sync/checkpoint [no test files]
```

```
? github.com/OffchainLabs/prysm/v7/cmd/client-stats [no test files]
? github.com/OffchainLabs/prysm/v7/cmd/client-stats/flags [no test files]
? github.com/OffchainLabs/prysm/v7/cmd/flags [no test files]
? github.com/OffchainLabs/prysm/v7/cmd/mock [no test files]
? github.com/OffchainLabs/prysm/v7/cmd/prysmctl [no test files]
? github.com/OffchainLabs/prysm/v7/cmd/prysmctl/checkpointsync [no test files]
? github.com/OffchainLabs/prysm/v7/cmd/prysmctl/db [no test files]
? github.com/OffchainLabs/prysm/v7/cmd/prysmctl/p2p [no test files]
ok github.com/OffchainLabs/prysm/v7/cmd/prysmctl/testnet 0.254s
ok github.com/OffchainLabs/prysm/v7/cmd/prysmctl/validator 0.119s
? github.com/OffchainLabs/prysm/v7/cmd/prysmctl/weaksubjectivity [no test files]
ok github.com/OffchainLabs/prysm/v7/cmd/validator 0.110s
ok github.com/OffchainLabs/prysm/v7/cmd/validator/accounts 15.916s
? github.com/OffchainLabs/prysm/v7/cmd/validator/db [no test files]
ok github.com/OffchainLabs/prysm/v7/cmd/validator/flags 0.022s
ok github.com/OffchainLabs/prysm/v7/cmd/validator/slashing-protection 0.156s
ok github.com/OffchainLabs/prysm/v7/cmd/validator/wallet 0.604s
? github.com/OffchainLabs/prysm/v7/cmd/validator/web [no test files]
ok github.com/OffchainLabs/prysm/v7/config/params 49.790s
FAIL
```

8.2 Automated Tools

8.2.1 AuditAgent

The AuditAgent is an AI-powered smart contract auditing tool that analyses code, detects vulnerabilities, and provides actionable fixes. It accelerates the security analysis process, complementing human expertise with advanced AI models to deliver efficient and comprehensive smart contract audits. Available at <https://app.auditagent.nethermind.io>.

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.